APB Protocol using Verilog without wait state

```
//
module apb_slave (
    input  logic        pclk,
    input  logic        rst_n,
    input  logic [31:0]  paddr,
    input  logic        psel,

    input  logic        penable,
    input  logic        pwrite,
    input  logic [31:0]  pwdata,

    output logic [31:0]  prdata,
    output logic         pready
);

  // Internal memory and states
  logic [31:0] mem [0:255];
  logic [1:0]  apb_st;

  // State encoding
  localparam logic [1:0]
    SETUP    = 2'b00,
    W_ENABLE = 2'b01,
    R_ENABLE = 2'b10;
```

```
 // Sequential logic
always_ff @(posedge pclk or negedge rst_n) begin
 if (!rst_n) begin
   apb_st <= SETUP;
   prdata <= 0;
   pready <= 1;
   for (int i = 0; i < 256; i++)
     mem[i] <= i;  // initialize memory for demo
 end
 else begin
   case (apb_st)

     //================================
     // SETUP phase: prepare to read or write
     //================================
     SETUP: begin
       prdata <= 0;
       if (psel && !penable) begin
         if (pwrite)
           apb_st <= W_ENABLE;
         else begin
           apb_st <= R_ENABLE;
           prdata <= mem[paddr]; // return stored data
         end
       end
```

```
        end
       //================================
       // WRITE ENABLE phase
       //================================

        W_ENABLE: begin
          if (psel && penable && pwrite) begin
            mem[paddr] <= pwdata; // perform write
          end
          apb_st <= SETUP; // return to idle
        end

        //================================
        // READ ENABLE phase
        //================================
        R_ENABLE: begin
          apb_st <= SETUP; // read already done in previous state
        end

      endcase
    end
  end

endmodule
module tb_apb;
```

```
  logic        pclk;
  logic        rst_n;
  logic [31:0]  paddr;
  logic        psel;
  logic        penable;
  logic        pwrite;
  logic [31:0]  pwdata;
  logic [31:0]  prdata;
  logic        pready;


  // Instantiate DUT
  apb_slave dut
(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pwrite
),.pwdata(pwdata),.prdata(prdata),.pready(pready) );
initial begin
    pclk = 0;
    forever #5 pclk = ~pclk;
  end
initial begin
    rst_n = 0;
    #20 rst_n = 1;
  end


 // APB write task
  task automatic apb_write(input [31:0] addr, input [31:0] data);
```

```
  @(posedge pclk);
  paddr   = addr;
  pwrite  = 1;
  pwdata  = data;
  psel    = 1;
  penable = 0;

  @(posedge pclk);
  penable = 1;
  @(posedge pclk);
  psel    = 0;
  penable = 0;
  pwrite  = 0;

  $display("[%0t] WRITE: addr=%0d data=0x%0h", $time, addr, data);
 endtask

task automatic apb_read(input [31:0] addr, output [31:0] data);
  @(posedge pclk);
  paddr   = addr;
  pwrite  = 0;
  psel    = 1;
  penable = 0;

  @(posedge pclk);
```

```
    penable = 1;

    @(posedge pclk);
    data = prdata;
    psel    = 0;
    penable = 0;
    $display("[%0t] READ : addr=%0d data=0x%0h", $time, addr, data);
  endtask

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars("tb_apb",0);
end

 // Stimulus
  logic [31:0] rdata;
  initial begin
    psel = 0; penable = 0; pwrite = 0; paddr = 0; pwdata = 0;
    wait(rst_n == 1);
    #10;

    apb_write(32'd10, 32'hAABB_CCDD);
    #10;
    apb_read(32'd10, rdata);
```

```
    if (rdata == 32'hAABB_CCDD)
      $display("[%0t]  READBACK SUCCESS", $time);
    else
      $display("[%0t]  READBACK FAIL: got %h", $time, rdata);

    #20;
    apb_read(32'd5, rdata);
    $display("[%0t] READ addr=5 got=%0d", $time, rdata);

    #50 $finish;
  end
endmodule
```

Simulation Command

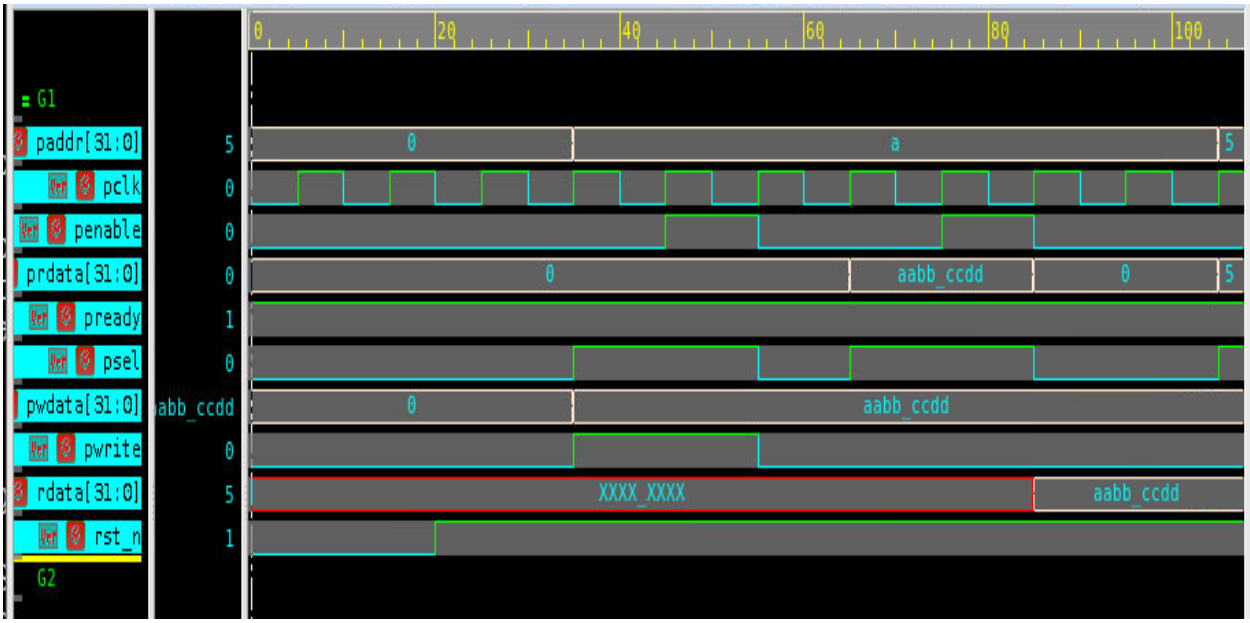
```
[b23n_sarif_727@prac3 apb]$ vcs -sverilog apb.v -kdb -lca -debug_access+all -full64
```

Simulation results

```
[55] WRITE: addr=10 data=0xaabbccdd
[85] READ : addr=10 data=0xaabbccdd
[85]  READBACK SUCCESS
[125] READ : addr=5 data=0x5
[125] READ addr=5 got=5
$finish called from file "apb.v", line 159.
$finish at simulation time                 175
           V C S   S i m u l a t i o n   R e p o r t
Time: 175
```

CPU Time: 0.090 seconds; Data structure size: 0.0Mb

Wed Jul 22 20:55:55 2026



APB Slave with PLVERR and wait-state N=2

With Wait-state

//

```
module apb_slave #(parameter WAIT_CYCLES = 2)(
        input  logic         pclk,
      input  logic         rst_n,
      input  logic [31:0]  paddr,
      input  logic         psel,
      input  logic         penable,
      input  logic         pwrite,
      input  logic [31:0]  pwdata,
        output logic [31:0]  prdata,
        output logic         pready,
```

```
        output logic        pslverr
);

logic [31:0] mem [0:255];
logic [1:0] apb_st;
localparam SETUP    = 2'b00, W_ENABLE = 2'b01,R_ENABLE = 2'b10;
integer wait_count;

always_ff @(posedge pclk or negedge rst_n) begin
        if (!rst_n) begin
                apb_st     <= SETUP;
                prdata     <= 32'd0;
            pready     <= 1'b1;
            pslverr    <= 1'b0;
            wait_count <= 0;

                for (int i = 0; i < 256; i++)
                        mem[i] <= i;
        end
        else begin
                case (apb_st)

                        SETUP: begin
                                prdata  <= 32'd0;
                         pslverr <= 1'b0;
```

```
 pready  <= 1'b0;

        if (psel && !penable) begin
                if (paddr > 255) begin
                        pslverr <= 1'b1;
                        pready  <= 1'b1;
                        if (pwrite)
                                apb_st <= W_ENABLE;
                        else
                                apb_st <= R_ENABLE;
                end
                else begin
                        wait_count <= WAIT_CYCLES;
                        if (pwrite)
                                apb_st <= W_ENABLE;
                        else
                                apb_st <= R_ENABLE;
                end
        end
end

W_ENABLE: begin
        if (psel && penable) begin
                if (wait_count > 0) begin
                        pready     <= 1'b0;
```

```
                wait_count <= wait_count - 1;
            end
            else begin
                pready <= 1'b1;
                if (paddr <= 255)
                    mem[paddr] <= pwdata;
                apb_st <= SETUP;
            end
        end
    end
    R_ENABLE: begin
        if (psel && penable) begin
            if (wait_count > 0) begin
                pready     <= 1'b0;
                wait_count <= wait_count - 1;
            end
            else begin
                pready <= 1'b1;
                if (paddr <= 255)
                    prdata <= mem[paddr];
                apb_st <= SETUP;
            end
        end
    end
    default: apb_st <= SETUP;
```

```
            endcase
        end
end
endmodule
```

Testbench

```
//
module tb_apb;
logic       pclk;
logic       rst_n;
logic [31:0] paddr;
logic       psel;
logic       penable;
logic       pwrite;
logic [31:0] pwdata;
logic [31:0] prdata;
logic       pready;
logic       pslverr;
logic [31:0] rdata;


apb_slave #(.WAIT_CYCLES(2))
dut(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pw
rite),.pwdata(pwdata),.prdata(prdata),.pready(pready),.pslverr(pslverr));


initial begin
        pclk = 0;
        forever #5 pclk = ~pclk;
```

```
end

initial begin
        rst_n = 0;
        #20;
        rst_n = 1;
end

task automatic apb_write(input [31:0] addr,input [31:0] data);
begin
        @(posedge pclk);
        paddr   <= addr;
       pwdata  <= data;
       pwrite  <= 1'b1;
       psel    <= 1'b1;
       penable <= 1'b0;
        $display("[%0t] WRITE SETUP phase : addr=%0d data=%h",$time, addr,
data);

        @(posedge pclk);
        penable <= 1'b1;
        $display("[%0t] WRITE ACCESS phase: PREADY=%b",$time, pready);

while (!pready)
begin
        @(posedge pclk);
```

```
        $display("[%0t] WRITE WAIT-state : PREADY=%b",$time, pready);
end

@(posedge pclk);
$display("[%0t] WRITE DONE-state : addr=%0d data=%h
PSLVERR=%b",$time, addr, data, pslverr);
psel    <= 1'b0;
penable <= 1'b0;
pwrite  <= 1'b0;
end
endtask

task automatic apb_read(input  [31:0] addr,output [31:0] data);
begin
        @(posedge pclk);
        paddr   <= addr;
      pwrite  <= 1'b0;
      psel    <= 1'b1;
      penable <= 1'b0;
        $display("[%0t] READ SETUP  : addr=%0d",$time, addr);

        @(posedge pclk);
        penable <= 1'b1;
        $display("[%0t] READ ACCESS : PREADY=%b", $time, pready);

while (!pready)
```

```
begin
        @(posedge pclk);
        $display("[%0t] READ WAIT   : PREADY=%b",$time, pready);
end

@(posedge pclk);
data = prdata;
$display("[%0t] READ DONE   : addr=%0d data=%h PSLVERR=%b",$time,
addr, data, pslverr);
psel    <= 1'b0;
penable <= 1'b0;
end
endtask

initial begin
          psel    = 0;
       penable = 0;
       pwrite  = 0;
       paddr   = 0;
       pwdata  = 0;

          wait(rst_n == 1);
          #10;
          apb_write(32'd10, 32'hAABB_CCDD);
          apb_read(32'd10, rdata);
```

```
        if (rdata == 32'hAABB_CCDD) begin
          $display("[%0t] READBACK SUCCESS", $time);
      end
      else begin
          $display("[%0t] READBACK FAIL: got=%h", $time, rdata);
      end

        apb_write(32'd300, 32'h1234_5678);
        if (pslverr)
                $display("[%0t] INVALID ADDRESS: PSLVERR=1", $time);
        else
                $display("[%0t] ERROR: PSLVERR=0", $time);
        #20;
        $finish;
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,tb_apb);
end
endmodule
```

Simulation Command

[b23n_sarif_727@prac3 apb_protocol]$ vcs -sverilog apb_design.sv apb_tb.sv -kdb -lca -debug_access+all -full64 && ./simv

[b23n_sarif_727@prac3 apb_protocol]$ ./simv -verdi

Simulation results

[35] WRITE SETUP phase : addr=10 data=aabbccdd

[45] WRITE ACCESS phase: PREADY=0

[55] WRITE WAIT-state : PREADY=0

[65] WRITE WAIT-state : PREADY=0

[75] WRITE WAIT-state : PREADY=0

[85] WRITE WAIT-state : PREADY=1

[95] WRITE DONE-state : addr=10 data=aabbccdd PSLVERR=0

[105] READ SETUP : addr=10

[115] READ ACCESS : PREADY=0

[125] READ WAIT : PREADY=0

[135] READ WAIT : PREADY=0

[145] READ WAIT : PREADY=0

[155] READ WAIT : PREADY=1

[165] READ DONE : addr=10 data=00000000 PSLVERR=0

[165] READBACK FAIL: got=00000000

[175] WRITE SETUP phase : addr=300 data=12345678

[185] WRITE ACCESS phase: PREADY=0

[195] WRITE WAIT-state : PREADY=1

[205] WRITE DONE-state : addr=300 data=12345678 PSLVERR=1

[205] INVALID ADDRESS: PSLVERR=1

$finish called from file "apb_tb.sv", line 108.

$finish at simulation time 225

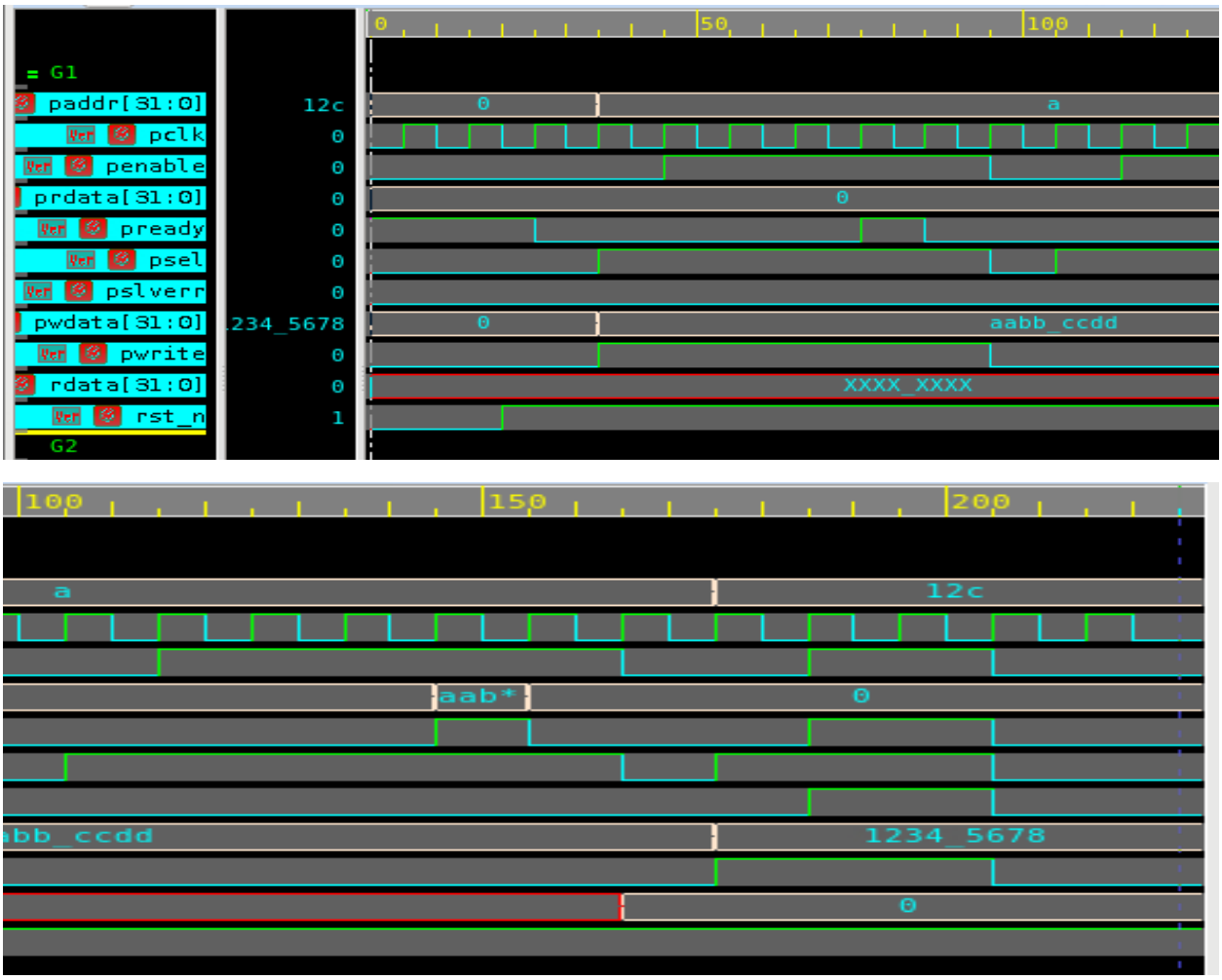
V C S S i m u l a t i o n R e p o r t

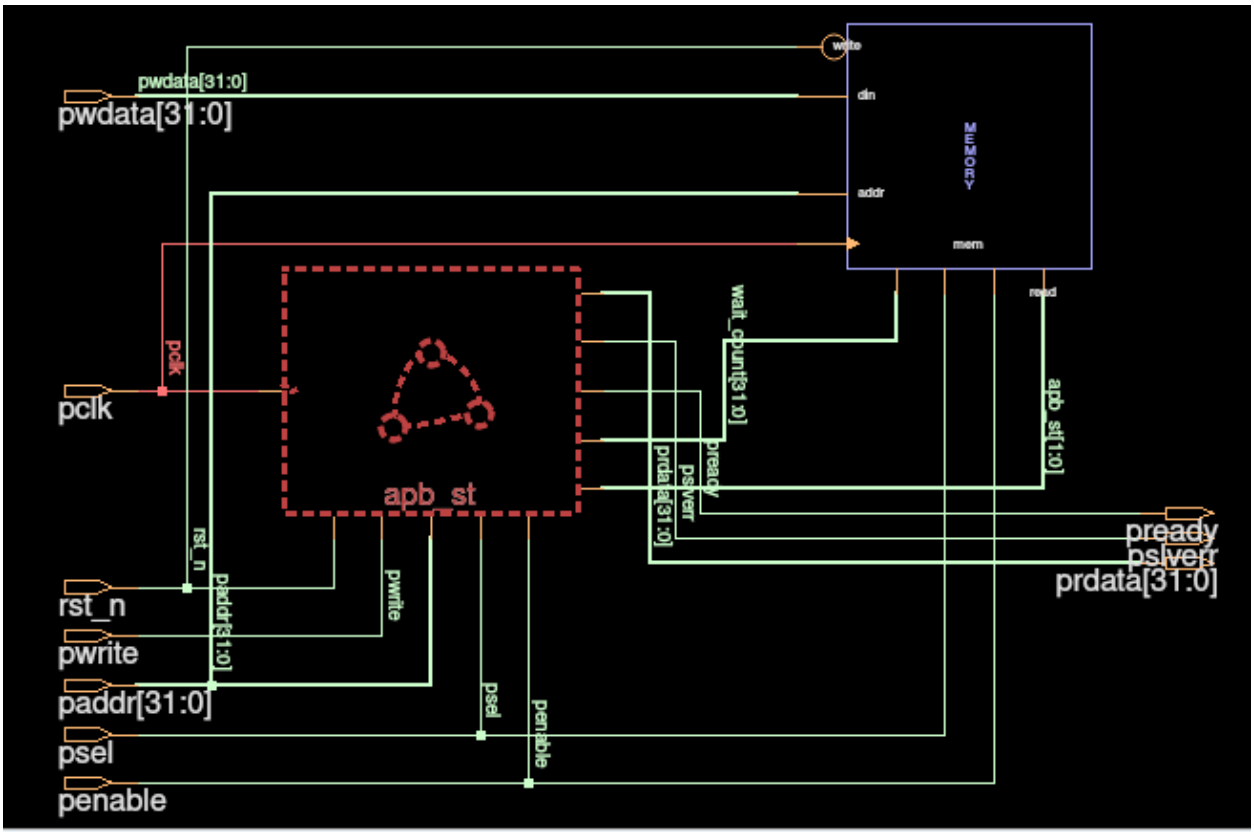
Time: 225

CPU Time: 0.100 seconds; Data structure size: 0.0Mb

Mon Aug 17 23:46:48 2026

Simulation waveform





Without Wait-state

```
//
module apb_slave (
        input  logic        pclk,
      input  logic        rst_n,
      input  logic [31:0] paddr,
      input  logic        psel,
      input  logic        penable,
      input  logic        pwrite,
      input  logic [31:0] pwdata,
      output logic [31:0] prdata,
      output logic        pready,
```

```
    output logic        pslverr
);

logic [31:0] mem [0:255];
logic [1:0] apb_st;
localparam logic [1:0] SETUP = 2'b00, W_ENABLE = 2'b01, R_ENABLE = 2'b10;

always_ff @(posedge pclk or negedge rst_n) begin
    if (!rst_n) begin
        apb_st  <= SETUP;
        prdata  <= 32'd0;
        pready  <= 1'b1;
        pslverr <= 1'b0;
        for (int i = 0; i < 256; i++)
            mem[i] <= i;
    end
    else begin
        case (apb_st)
            SETUP: begin
                prdata  <= 32'd0;
                pslverr <= 1'b0;
                pready  <= 1'b1;

                if (psel && !penable) begin
                    if (paddr > 255) begin
```

```
                pslverr <= 1'b1;
                apb_st <= SETUP;
            end
            else if (pwrite) begin
                apb_st <= W_ENABLE;
            end
            else begin
                apb_st <= R_ENABLE;
            end
        end
    end
    W_ENABLE: begin
        if (psel && penable && pwrite) begin
            mem[paddr] <= pwdata;
        end
        apb_st <= SETUP;
    end
    R_ENABLE: begin
        if (psel && penable && !pwrite) begin
            prdata <= mem[paddr];
        end
        apb_st <= SETUP;
    end
    default: begin
        apb_st  <= SETUP;
```

```
                    prdata  <= 32'd0;
                    pready  <= 1'b1;
                    pslverr <= 1'b0;
                end
            endcase
        end
end
endmodule
```

Testbench

```
//
module tb_apb;
logic        pclk;
logic        rst_n;
logic [31:0] paddr;
logic        psel;
logic        penable;
logic        pwrite;
logic [31:0] pwdata;
logic [31:0] prdata;
logic        pready;
logic        pslverr;


apb_slave dut
(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pwrite
),.pwdata(pwdata),.prdata(prdata),.pready(pready),.pslverr(pslverr));
```

```
initial pclk = 0;
always #5 pclk = ~pclk;

initial begin
        rst_n = 0;
        #20 rst_n = 1;
end

task apb_write (input [31:0] addr, input [31:0] data);
begin
        @(posedge pclk);
        paddr   = addr;
        pwdata  = data;
        pwrite  = 1'b1;
        psel    = 1'b1;
        penable = 1'b0;

        @(posedge pclk);
        if (pslverr) begin
                $display("[%0t] WRITE ERROR: Addr=%0d Data=%h
PSLVERR=%b", $time, addr, data, pslverr);
        end
        else begin
                penable = 1'b1;
                @(posedge pclk);
```

```
                $display("[%0t] WRITE: Addr=%0d Data=%h PSLVERR=%b",
$time, addr, data, pslverr);
        end
        psel    = 1'b0;
        penable = 1'b0;
        pwrite  = 1'b0;
end
endtask

task apb_read (input [31:0] addr);
begin
        @(posedge pclk);
        paddr   = addr;
      pwrite  = 1'b0;
      psel    = 1'b1;
      penable = 1'b0;
        @(posedge pclk);
        if (pslverr) begin
                $display("[%0t] READ ERROR: Addr=%0d Data=%h
PSLVERR=%b", $time, addr, prdata,pslverr);
        end
        else begin
                penable = 1'b1;
                @(posedge pclk);
                $display("[%0t] READ: Addr=%0d Data=%h PSLVERR=%b",
$time, addr, prdata, pslverr);
```

```
        end
        psel    = 1'b0;
        penable = 1'b0;
end
endtask

initial begin
        psel    = 1'b0;
        penable = 1'b0;
        pwrite  = 1'b0;
        paddr   = 32'd0;
        pwdata  = 32'd0;

        wait(rst_n == 1);
        #10;
      apb_write(32'd10, 32'hAABBCCDD);
      apb_read(32'd10);
      apb_write(32'd300, 32'h11112222);
      apb_read(32'd300);
        #30 $finish;
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,tb_apb);
end
```

endmodule

Run-command

[b23n_sarif_727@prac3 apb_slave]$ vcs -sverilog apb_slave.sv apb_tb.sv -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 apb_slave]$ ./simv

[b23n_sarif_727@prac3 apb_slave]$ ./simv -verdi

Simulation Results:

[55] WRITE: Addr=10 Data=aabbccdd PSLVERR=0

[85] READ: Addr=10 Data=aabbccdd PSLVERR=0

[105] WRITE ERROR: Addr=300 Data=11112222 PSLVERR=1

[125] READ ERROR: Addr=300 Data=00000000 PSLVERR=1

$finish called from file "apb_tb.sv", line 82.

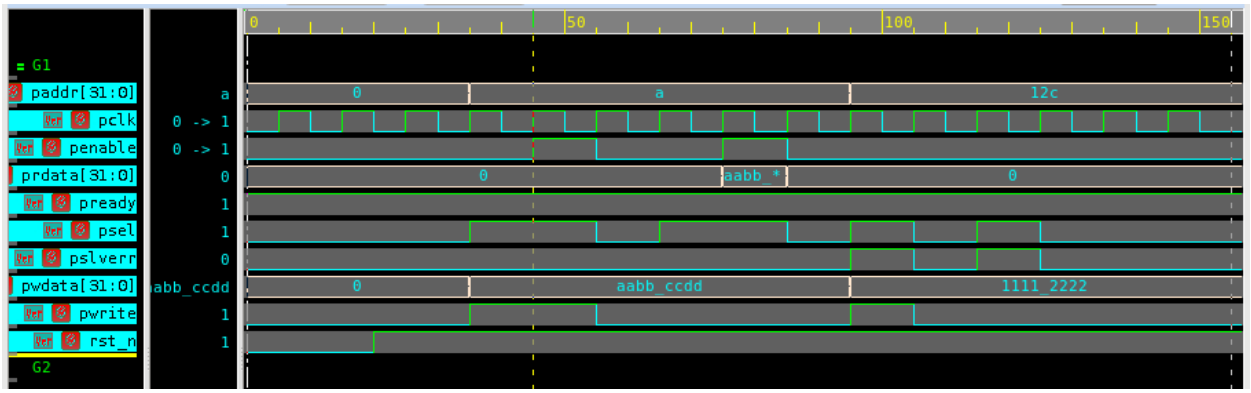
$finish at simulation time 155

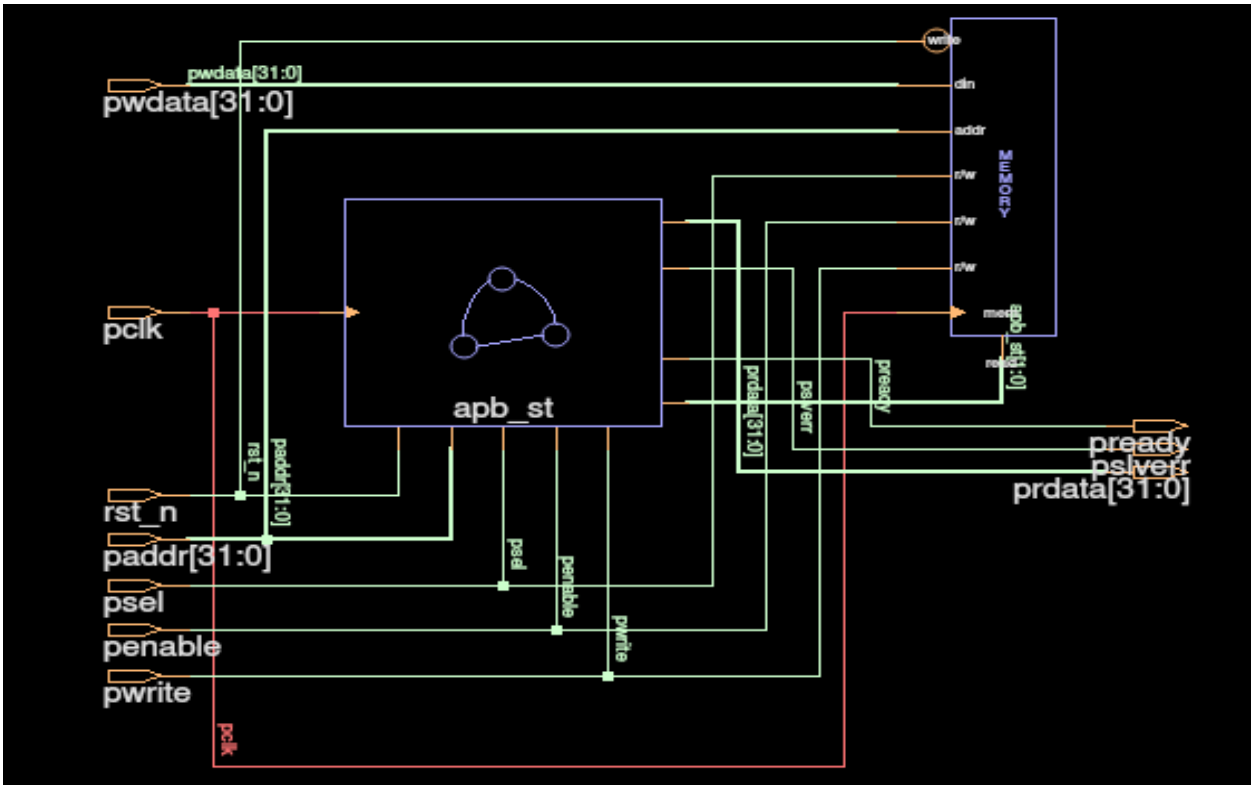
V C S S i m u l a t i o n R e p o r t

Time: 155

CPU Time: 0.090 seconds; Data structure size: 0.0Mb

Tue Aug 18 10:04:28 2026





APB Protocol Verification using Verilog with wait state N=2

//

```
module apb_slave #(parameter integer WAIT_CYCLES = 2)(
        input logic       pclk,
      input logic       rst_n,
      input logic [31:0] paddr,
      input logic       psel,
      input logic       penable,
      input logic       pwrite,
      input logic [31:0] pwdata,
        output logic [31:0] prdata,
```

```
    output logic        pready,
    output logic        pslverr
);

logic [31:0] mem [0:255];
typedef enum logic [1:0] { SETUP = 2'b00, W_ENABLE = 2'b01, R_ENABLE = 2'b10 } state_t;
state_t apb_st;
integer wait_count;

always_ff @(posedge pclk or negedge rst_n) begin
    if (!rst_n) begin
        apb_st     <= SETUP;
        prdata     <= 32'd0;
      pready     <= 1'b1;
      pslverr    <= 1'b0;
      wait_count <= 0;

        for (int i = 0; i < 256; i++)
            mem[i] <= i;
    end

    else begin
        case (apb_st)
            SETUP:
            begin
```

```
        pready  <= 1'b0;
        pslverr <= 1'b0;
        prdata  <= 32'd0;

        if (psel && !penable) begin
                if (paddr > 32'd255) begin
                        pslverr <= 1'b1;
                        pready  <= 1'b1;
                        apb_st <= SETUP;
                end
                else begin
                        wait_count <= WAIT_CYCLES;
                        if (pwrite)
                                apb_st <= W_ENABLE;
                        else
                                apb_st <= R_ENABLE;
                end
        end
end
W_ENABLE:
begin
        if (psel && penable && pwrite) begin
                if (wait_count > 0) begin
                        pready     <= 1'b0;
                        wait_count <= wait_count - 1;
```

```
                end
                else begin
                        pready <= 1'b1;
                        if (paddr <= 32'd255)
                                mem[paddr] <= pwdata;
                        apb_st <= SETUP;
                end
        end
end
R_ENABLE:
begin
        if (psel && penable && !pwrite) begin
                if (wait_count > 0) begin
                        pready     <= 1'b0;
                        wait_count <= wait_count - 1;
                end
                else begin
                        pready <= 1'b1;
                        if (paddr <= 32'd255)
                                prdata <= mem[paddr];
                        apb_st <= SETUP;
                end
        end
end
default:
```

```
                begin
                    apb_st     <= SETUP;
                    pready     <= 1'b1;
                    pslverr    <= 1'b0;
                    prdata     <= 32'd0;
                    wait_count <= 0;
                end
            endcase
        end
end
endmodule
```

Testbench

```
//
module tb_apb;
logic        pclk;
logic        rst_n;
logic [31:0] paddr;
logic        psel;
logic        penable;
logic        pwrite;
logic [31:0] pwdata;

logic [31:0] prdata;
logic        pready;
logic        pslverr;
```

```
apb_slave #(.WAIT_CYCLES(0))
dut(.pclk(pclk),.rst_n(rst_n),.paddr(paddr),.psel(psel),.penable(penable),.pwrite(pw
rite),.pwdata(pwdata),.prdata(prdata),.pready(pready),.pslverr(pslverr));


initial begin
        pclk = 1'b0;
        forever #5 pclk = ~pclk;
end


initial begin
        rst_n = 1'b0;
        #20;
        rst_n = 1'b1;
end


task automatic apb_write(input logic [31:0] addr, input logic [31:0] data);
begin
        @(posedge pclk);
        #1;
      paddr   = addr;
      pwdata  = data;
      pwrite  = 1'b1;
      psel    = 1'b1;
      penable = 1'b0;
```

```
    $display("[%0t] WRITE SETUP : ADDR=%0d DATA=%h", $time, addr, data);

    @(posedge pclk);
    #1;
    penable = 1'b1;
    $display("[%0t] WRITE ACCESS: PREADY=%b", $time, pready);

    while (!pready) begin
        @(posedge pclk);
        #1;
        $display("[%0t] WRITE WAIT  : PREADY=%b", $time, pready);
    end
    $display("[%0t] WRITE DONE  : ADDR=%0d DATA=%h PSLVERR=%b", $time, addr, data, pslverr);

    @(posedge pclk);
    #1;
    psel    = 1'b0;
    penable = 1'b0;
    pwrite  = 1'b0;
end
endtask

task automatic apb_read(input logic [31:0] addr, output logic [31:0] data);
begin
```

```
  @(posedge pclk);
#1;
paddr   = addr;
pwrite  = 1'b0;
psel    = 1'b1;
penable = 1'b0;
  $display("[%0t] READ SETUP  : ADDR=%0d", $time, addr);

  @(posedge pclk);
#1;
penable = 1'b1;
$display("[%0t] READ ACCESS : PREADY=%b", $time, pready);

while (!pready) begin
        @(posedge pclk);
        #1;
        $display("[%0t] READ WAIT : PREADY=%b", $time, pready);
end

data = prdata;
$display("[%0t] READ DONE : ADDR=%0d DATA=%h PSLVERR=%b",
$time, addr, data, pslverr);

@(posedge pclk);
#1;
psel    = 1'b0;
```

```
        penable = 1'b0;
end
endtask

logic [31:0] rdata;

initial begin
          psel    = 1'b0;
        penable = 1'b0;
        pwrite  = 1'b0;
        paddr   = 32'd0;
        pwdata  = 32'd0;
        rdata   = 32'd0;

        wait (rst_n == 1'b1);
        #10;

        $display("\n=====================================");
        $display("TEST 1: RESET");
        $display("=====================================");

        @(posedge pclk);
        #1;
          $display("[%0t] RESET CHECK: PREADY=%b PSLVERR=%b",$time,
pready, pslverr);
```

```
$display("\n=======================================");
$display("TEST 2: NORMAL WRITE");
$display("=======================================");

apb_write(32'd10, 32'hAABBCCDD);
$display("\n=======================================");
$display("TEST 3: NORMAL READ");
$display("=======================================");

apb_read(32'd10, rdata);
if (rdata == 32'hAABBCCDD)
   $display("READBACK PASS");
else
   $display("READBACK FAIL: DATA=%h", rdata);

$display("\n=======================================");
$display("TEST 4: WRITE WITH WAIT STATES");
$display("=======================================");

apb_write(32'd20, 32'h12345678);
$display("\n=======================================");
$display("TEST 5: READ WITH WAIT STATES");
$display("=======================================");

apb_read(32'd20, rdata);
```

```
        if (rdata == 32'h12345678)
            $display("READBACK PASS");
        else
            $display("READBACK FAIL: DATA=%h", rdata);

        $display("\n=======================================");
        $display("TEST 6: INVALID WRITE");
        $display("=======================================");

        apb_write(32'd300, 32'hDEADBEEF);
        $display("\n=======================================");
        $display("TEST 7: INVALID READ");
        $display("=======================================");

        apb_read(32'd300, rdata);
        #20;
        $display("\n=======================================");
        $display("ALL TESTS COMPLETED");
        $display("=======================================");

        $finish;
    end
    initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb_apb);
```

```
end

endmodule
```

Simulation waveform

```
========================================

TEST 1: RESET

========================================

[36] RESET CHECK: PREADY=0 PSLVERR=0


========================================

TEST 2: NORMAL WRITE

========================================

[46] WRITE SETUP : ADDR=10 DATA=aabbccdd

[56] WRITE ACCESS: PREADY=0

[66] WRITE WAIT  : PREADY=1

[66] WRITE DONE  : ADDR=10 DATA=aabbccdd PSLVERR=0


========================================

TEST 3: NORMAL READ

========================================

[86] READ SETUP  : ADDR=10

[96] READ ACCESS : PREADY=0

[106] READ WAIT : PREADY=1

[106] READ DONE : ADDR=10 DATA=aabbccdd PSLVERR=0

READBACK PASS
```

==========

TEST 4: WRITE WITH WAIT STATES

==========

[126] WRITE SETUP : ADDR=20 DATA=12345678

[136] WRITE ACCESS: PREADY=0

[146] WRITE WAIT : PREADY=1

[146] WRITE DONE : ADDR=20 DATA=12345678 PSLVERR=0

==========

TEST 5: READ WITH WAIT STATES

==========

[166] READ SETUP : ADDR=20

[176] READ ACCESS : PREADY=0

[186] READ WAIT : PREADY=1

[186] READ DONE : ADDR=20 DATA=12345678 PSLVERR=0

READBACK PASS

==========

TEST 6: INVALID WRITE

==========

[206] WRITE SETUP : ADDR=300 DATA=deadbeef

[216] WRITE ACCESS: PREADY=1

[216] WRITE DONE : ADDR=300 DATA=deadbeef PSLVERR=1

==========

TEST 7: INVALID READ

==========

[236] READ SETUP : ADDR=300

[246] READ ACCESS : PREADY=1

[246] READ DONE : ADDR=300 DATA=00000000 PSLVERR=1

==========

ALL TESTS COMPLETED

==========

$finish called from file "apb_tb.sv", line 156.

$finish at simulation time   276

V C S S i m u l a t i o n R e p o r t

Time: 276

CPU Time: 0.090 seconds;   Data structure size: 0.0Mb

Wed Aug 19 13:51:42 2026